

基于 DolphinDB + vn.py 的高频量化交易系统

demo课题汇报文档

一、项目概述

1.1 课题目标

构建一个端到端的高频量化交易系统，实现从实时行情采集、因子计算、信号生成到订单执行的完整闭环。重点学习和实践以下技术：

- DolphinDB**：高性能时序数据库的流式计算能力
- vn.py**：国内主流量化交易框架的架构设计
- 消息队列**：RabbitMQ 实现系统解耦

1.2 系统功能

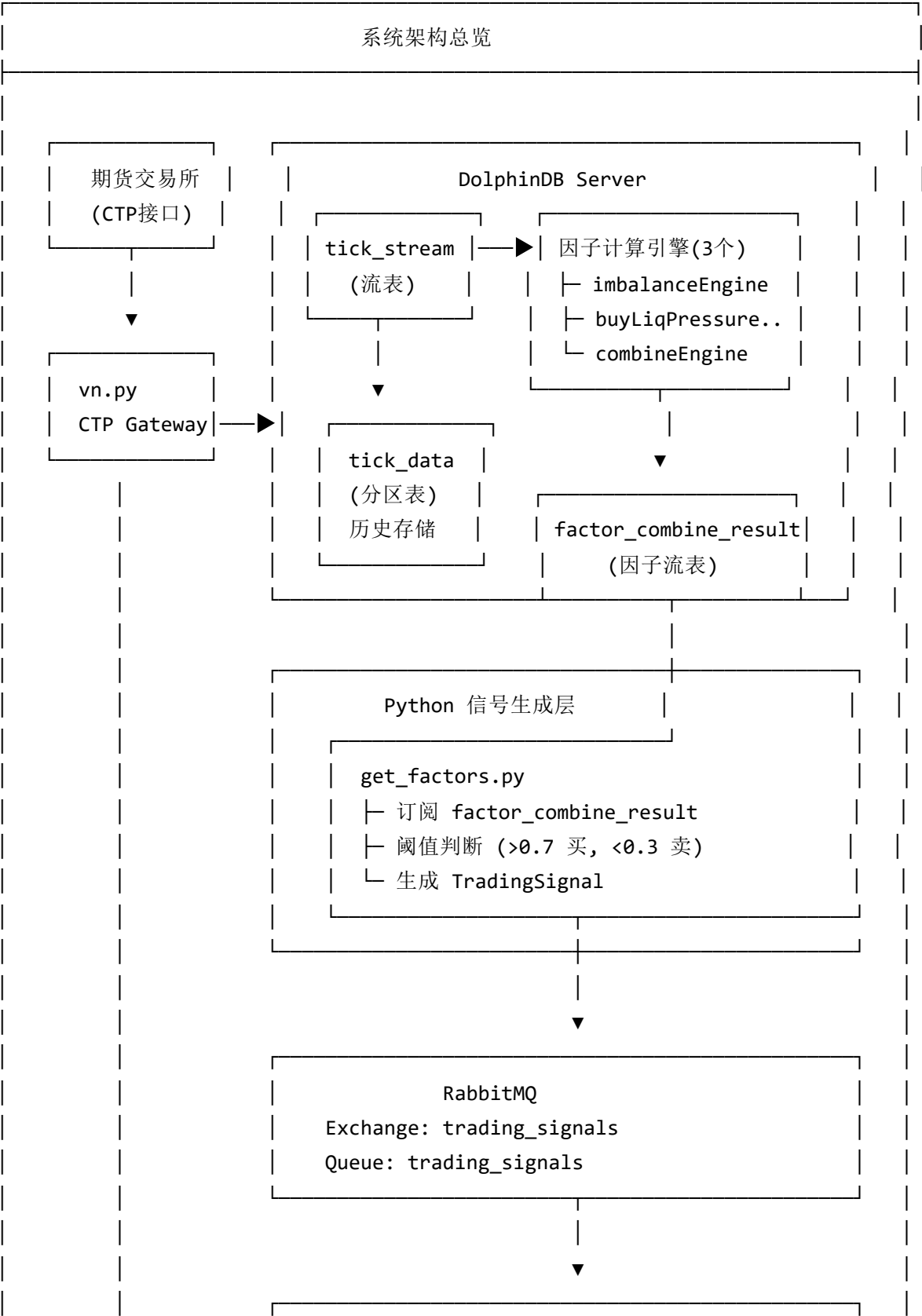
模块	功能描述	技术实现
数据采集	实时 tick 数据获取	vn.py CTP Gateway
数据存储	流式写入 + 分区持久化	DolphinDB 流表/分区表
因子计算	多因子实时流式计算	DolphinDB 响应式状态引擎
信号生成	因子阈值触发交易信号	Python + DolphinDB 订阅
信号分发	异步可靠的信号推送	RabbitMQ 消息队列
订单执行	接收信号自动下单	vn.py 自定义 Engine

1.3 核心亮点

- 流批一体**：DolphinDB 统一处理实时流数据和历史批数据
- 事件驱动**：vn.py 事件引擎实现高效异步处理
- 系统解耦**：MQ 分离信号生成和订单执行，提高系统可靠性
- 对手价下单**：无价格信号自动获取实时对手价，确保成交

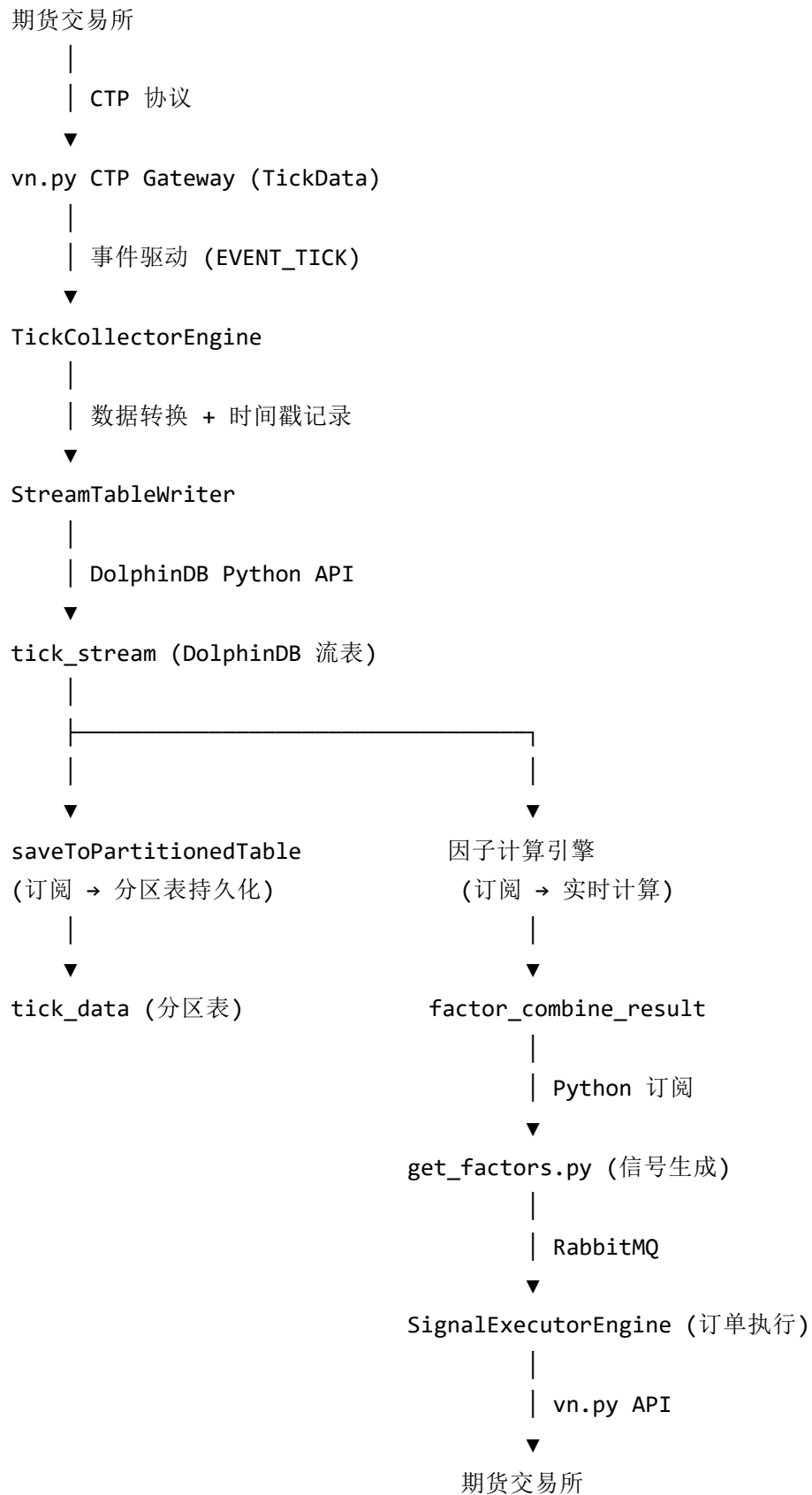
二、系统架构

2.1 整体架构图





2.2 数据流向



三、项目结构

```
imba-vnpy/
├─ myvnpy/                                # 自定义 vn.py 扩展模块
│   └─ app/
│       └─ data_recorder/                # 数据采集模块
│           ├── tick_collector_engine.py  # Tick 数据采集引擎
│           ├── tick_data_converter.py    # 数据格式转换器
│           ├── stream_table_writer.py    # DolphinDB 流表写入器
│           └─ dolphindb_session.py       # DolphinDB 会话管理
│       └─ signal_executor/              # 信号执行模块
│           └─ signal_executor_engine.py  # 信号执行引擎
│   └─ gateway/
│       └─ ctp/
│           ├── ctp_gateway.py            # 修复版 CTP 网关
│           └─ ctp_patch.py               # CTP 补丁
│   └─ mq/                                # 消息队列模块
│       ├── connection.py                # RabbitMQ 连接管理
│       ├── producer.py                  # 信号生产者
│       └─ consumer.py                   # 信号消费者
│   └─ signal/
│       └─ signal.py                     # 交易信号数据结构
├─ script/
│   └─ dos/                              # DolphinDB 脚本
│       ├── init.dos                    # 初始化入口脚本
│       ├── creat_table/
│           ├── creat_table.dos          # 分区表创建
│           └─ creat_stream_table.dos    # 流表创建
│       ├── factors/
│           ├── factor1.dos              # 买卖不平衡因子
│           ├── factor2.dos              # 买方流动性压力因子
│           ├── factor3.dos              # 组合因子
│           └─ README.txt                 # 因子说明文档
│       └─ lag_cacul/                    # 延迟统计脚本
└─ python/
    └─ get_factors.py                    # 因子订阅 + 信号生成
```

```
|
├─ run.py           # 主程序入口
└─ README.md       # 项目文档
```

四、核心模块实现

4.1 DolphinDB 数据存储层

4.1.1 分区表设计

```
// 分区策略：按日期 VALUE 分区 + 按合约 HASH 分区
db = database(
    directory="dfs://futures_snapshot",
    partitionType=COMPO,
    partitionScheme=[
        database(, VALUE, 2024.01.01..2030.12.31), // 日期分区
        database(, HASH, [SYMBOL, 10])           // 合约哈希分区
    ]
)
```

设计考量：

- **日期分区**：便于按时间范围查询历史数据
- **哈希分区**：分散热点，提高并发写入性能
- **组合分区**：兼顾查询效率和写入性能

4.1.2 流表 + 订阅机制

```
// 创建共享流表
share(streamTable(1000000:0, colNames, colTypes), `tick_stream)

// 订阅流表，自动持久化到分区表
subscribeTable(
    tableName="tick_stream",
    actionName=`save_to_partitioned_table,
    handler=saveToPartitionedTable,
    msgAsTable=true,
    batchSize=10,
    throttle=0.05
)
```

核心参数说明：

参数	值	说明
batchSize	10	累积 10 条触发一次回调
throttle	0.05	最多 50ms 触发一次
msgAsTable	true	以表格形式传递数据

4.2 因子计算引擎

4.2.1 因子1： 买卖不平衡因子 (Imbalance)

```
// 无状态因子 - 每条数据独立计算
def imbalance(bid_volume_1, ask_volume_1) {
    return iif(bid_volume_1 + ask_volume_1 == 0, 0,
               (bid_volume_1 - ask_volume_1) / (bid_volume_1 + ask_volume_1))
}
```

因子特性：

- 输出范围： [-1, 1]
- +1 表示完全买方主导， -1 表示完全卖方主导
- 无状态，实时反映瞬时订单簿力量对比

4.2.2 因子2：买方流动性压力因子 (Buy Liquidity Pressure)

```
// 有状态因子 - 需要维护历史窗口
@state
def buyLiquidityPressure(bid_volume_1, ask_volume_1, ask_price_1, bid_price_1) {
    bid_volume_1_ma = mavg(bid_volume_1, 5*60) // 5分钟移动平均
    ask_volume_1_ma = mavg(ask_volume_1, 5*60)
    buy_prop = bid_volume_1_ma / (bid_volume_1_ma + ask_volume_1_ma)

    spd_ma = mavg(ask_price_1 - bid_price_1, 5*60) // 平均价差

    return iif(spd_ma == 0, 0, buy_prop / spd_ma)
}
```

因子特性：

- 输出范围：[0, +∞)
- 值越高表示买方力量强且流动性好
- 有状态，使用 @state 装饰器维护滚动窗口

4.2.3 因子3：组合因子 (Combined Score)

```
@state
def combineFactor(bid_volume_1, ask_volume_1, ask_price_1, bid_price_1) {
    // 计算两个基础因子
    factor1 = imbalance(bid_volume_1, ask_volume_1)
    factor2 = buyLiquidityPressure(...)

    // 标准化到 [0, 1]
    imb_norm = (factor1 + 1) / 2
    blp_norm = min(factor2, 1)

    // 加权组合
    combined_score = 0.4 * imb_norm + 0.6 * blp_norm
    return combined_score
}
```

组合策略：

- 趋势因子(factor2)权重 0.6，更稳定
- 瞬时因子(factor1)权重 0.4，更敏感
- 输出范围：[0, 1]

4.2.4 响应式状态引擎

```
engine = createReactiveStateEngine(  
    name="combineFactorEngine",  
    metrics=<[datetime, combineFactor(bid_volume_1, ask_volume_1, ask_price_1, bid_price_1)]>,  
    dummyTable=inputSchema,  
    outputTable=outputTable,  
    keyColumn="symbol"  
)
```

关键点：

- keyColumn="symbol"：按合约分别维护状态
- 自动管理每个合约的历史数据窗口
- 新数据进入自动触发增量计算

4.3 信号生成模块

```
class FactorSubscriber:  
    def generate_signal(self, symbol: str, factor_value: float) -> TradingSignal:  
        # 阈值策略: > 0.7 做多, < 0.3 做空  
        if factor_value > 0.7:  
            signal_type = SignalType.BUY  
        elif factor_value < 0.3:  
            signal_type = SignalType.SELL  
        else:  
            return None # 中性区间不发信号  
  
        return TradingSignal(  
            signal_id=str(uuid.uuid4()),  
            symbol=symbol,  
            signal_type=signal_type,  
            factor_value=factor_value,  
            volume=1  
)
```

4.4 订单执行引擎

```
class SignalExecutorEngine(BaseEngine):
    def _get_price(self, signal: TradingSignal, use_ask: bool) -> float:
        """获取对手价"""
        if signal.price:
            return signal.price

        # 从实时行情获取对手价
        tick = self.main_engine.get_tick(vt_symbol)
        if tick:
            return tick.ask_price_1 if use_ask else tick.bid_price_1
        return None

    def execute_buy(self, signal: TradingSignal):
        price = self._get_price(signal, use_ask=True) # 买入用卖一价
        req = OrderRequest(
            symbol=symbol,
            exchange=exchange,
            direction=Direction.LONG,
            type=OrderType.LIMIT,
            offset=Offset.OPEN,
            price=price,
            volume=signal.volume
        )
        self.main_engine.send_order(req, self.gateway_name)
```

对手价规则：

操作	使用价格	原因
买入开多	ask_price_1 (卖一价)	吃掉卖盘
卖出开空	bid_price_1 (买一价)	吃掉买盘
平多	bid_price_1 (买一价)	吃掉买盘
平空	ask_price_1 (卖一价)	吃掉卖盘

五、DolphinDB 学习总结

5.1 流式计算核心概念

5.1.1 流表 (Stream Table)

定义：流表是 DolphinDB 中用于接收实时数据的内存表，支持订阅机制。

特点：

- 内存存储，读写速度极快
- 支持多订阅者同时消费
- 可设置容量上限，自动清理旧数据

个人理解：

流表本质上是一个"数据管道入口"，数据进入流表后可以触发多个下游处理逻辑（持久化、计算、转发等），这种设计实现了数据流的"一写多读"，非常适合实时系统。

5.1.2 响应式状态引擎 (Reactive State Engine)

定义：一种流计算引擎，自动维护每个 key 的历史状态，支持增量计算。

关键配置：

- `keyColumn`：分组字段，按此字段分别维护状态
- `metrics`：计算表达式，定义输出内容
- `@state` 装饰器：标记有状态函数

个人理解：

响应式状态引擎解决了流计算中的"状态管理"难题。传统流计算需要手动维护窗口数据，而 DolphinDB 通过 `@state` 装饰器和引擎自动管理，开发者只需关注计算逻辑本身。这种抽象大大降低了开发复杂度。

5.1.3 订阅机制 (Subscribe)

```
subscribeTable(  
    tableName="tick_stream",  
    actionName=`calc_imbalance`,  
    handler=imbalanceHandler, // 回调函数  
    msgAsTable=true,  
    batchSize=10,  
    throttle=0.05  
)
```

个人理解：

DolphinDB 的订阅机制类似于消息队列的消费者模式，但更加轻量。batchSize 和 throttle 参数可以在"实时性"和"吞吐量"之间灵活权衡：

- 高频场景：小 batchSize + 短 throttle
- 批量场景：大 batchSize + 长 throttle

5.2 有状态 vs 无状态因子

对比项	无状态因子	有状态因子
装饰器	无	@state
计算依赖	仅当前数据	历史窗口数据
典型函数	iif , 四则运算	mavg , msum , mstd
预热期	无	需要积累窗口数据
适用场景	瞬时指标	趋势指标

个人理解：

有状态因子的核心价值在于能够捕捉"趋势"信息。在实际应用中，我发现瞬时因子（无状态）噪音较大，而趋势因子（有状态）更加平滑稳定。两者组合使用可以互补：瞬时因子提供入场时机，趋势因子确认方向。

5.3 分区表设计原则

学习到的设计原则：

1. **分区粒度**：单个分区数据量建议 100MB-1GB
2. **分区字段选择**：优先选择查询条件中的高频字段
3. **组合分区**：多维度查询场景使用 COMPO 分区

个人理解：

分区表的本质是"空间换时间"——通过物理隔离数据，查询时只扫描必要的分区。在设计时需要预判查询模式：如果经常按日期+合约查询，就使用日期+合约组合分区；如果只按日期查询，单独日期分区即可。

5.4 遇到的问题与解决

问题1：订阅引擎列数不匹配

现象： Invalid input message. Expect 4 columns, but actually 23 columns

原因： 直接将 tick_stream 的全部 23 列传给引擎，但引擎只需要 4 列

解决方案： 添加包装函数（handler）进行列筛选

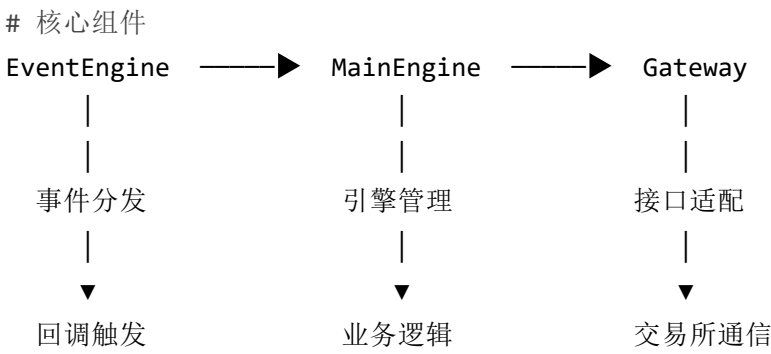
```
def imbalanceHandler(mutable msg) {  
    engine = getStreamEngine("imbalanceEngine")  
    selectedData = select symbol, datetime, bid_volume_1, ask_volume_1 from msg  
    engine.append!(selectedData)  
}
```

个人总结：

DolphinDB 的流计算引擎对输入 schema 有严格校验，这是一个类型安全的设计。虽然增加了一点开发成本，但能在运行时及早发现问题。

六、vn.py 学习总结

6.1 事件驱动架构



个人理解：

vn.py 的事件驱动架构是整个框架的核心。所有组件通过 EventEngine 解耦：Gateway 发布行情/订单事件，Engine 订阅并处理。这种设计让各模块可以独立开发和测试，非常适合量化系统的模块化需求。

6.2 自定义 Engine 开发

```
class SignalExecutorEngine(BaseEngine):
    engine_name = "SignalExecutor"

    def __init__(self, main_engine: MainEngine, event_engine: EventEngine, ...):
        super().__init__(main_engine, event_engine, self.engine_name)
        # 初始化自定义组件

    def start(self):
        """启动引擎"""
        pass

    def stop(self):
        """停止引擎"""
        pass
```

开发要点：

- 1. 继承 BaseEngine ，实现标准接口
- 2. 通过 main_engine 访问其他引擎和 Gateway

3. 通过 event_engine 订阅/发布事件

个人理解：

`vn.py` 的 Engine 扩展机制非常灵活。通过继承 `BaseEngine` 并注册到 `MainEngine`，自定义功能可以无缝融入框架。在本项目中，我开发了两个自定义 Engine：

- `TickCollectorEngine`：数据采集
- `SignalExecutorEngine`：信号执行

这种设计让业务逻辑与框架核心保持分离，便于维护和扩展。

在实际的大规模的场景中考虑可扩展性，应该设计成分布式/弹性扩展/容灾设计等方式考虑，也能兼容多账户。

6.3 Gateway 机制

Gateway 职责：

- 封装交易所通信协议
- 转换数据格式（交易所格式 ↔ `vn.py` 格式）
- 处理连接管理和重连

个人理解：

Gateway 是 `vn.py` 与交易所的连通器。不同交易所不同的协议和数据格式，Gateway 将这些差异屏蔽掉，让上层代码可以统一处理。在本项目中使用的 CTP Gateway 是期货领域最常用的接口。

6.4 订单管理

```
req = OrderRequest(  
    symbol="IM2603",  
    exchange=Exchange.CFFEX,  
    direction=Direction.LONG,      # 多/空  
    type=OrderType.LIMIT,          # 限价/市价  
    offset=Offset.OPEN,            # 开仓/平仓  
    price=7500.0,  
    volume=1  
)  
order_id = main_engine.send_order(req, "CTP")
```

关键概念：

- Direction：多头 (LONG) / 空头 (SHORT)
- Offset：开仓 (OPEN) / 平仓 (CLOSE)
- OrderType：限价 (LIMIT) / 市价 (MARKET)

个人理解：

vn.py 的订单模型抽象得非常清晰。Direction 和 Offset 的组合可以表达所有交易意图：

- 开多：LONG + OPEN
- 开空：SHORT + OPEN
- 平多：SHORT + CLOSE
- 平空：LONG + CLOSE

七、消息队列集成

7.1 为什么使用 MQ

问题：信号生成和订单执行如果强耦合，会带来：

- 单点故障风险
- 难以水平扩展
- 调试困难

解决方案：引入 RabbitMQ 实现解耦

信号生成器 —► RabbitMQ —► 订单执行器
(生产者) (消息中转) (消费者)

7.2 消息结构设计

```
@dataclass
class TradingSignal:
    signal_id: str          # 唯一标识
    symbol: str             # 合约代码
    signal_type: SignalType # BUY/SELL/CLOSE_LONG/CLOSE_SHORT
    signal_strength: SignalStrength # 信号强度
    factor_value: float      # 因子值
    price: Optional[float]   # 建议价格（可选）
    volume: int              # 建议数量
    timestamp: datetime      # 生成时间
```

7.3 生产者-消费者模式

```
# 生产者 (get_factors.py)
producer = SignalProducer(connection)
producer.publish(signal) # 序列化 + 发送

# 消费者 (SignalExecutorEngine)
consumer = SignalConsumer(connection, callback=self.on_signal)
consumer.start() # 启动消费线程
```

个人理解：

MQ 的引入带来了几个好处：

1. **可靠性**：消息持久化，即使消费者崩溃也不丢失
2. **削峰**：高频信号可以在队列中缓冲
3. **扩展性**：可以增加多个消费者并行处理
4. **可观测性**：RabbitMQ 提供管理界面，方便监控

八、运行指南

8.1 环境准备

```
# 1. 安装 Python 依赖
pip install vnpy dolphindb pika pandas

# 2. 启动 DolphinDB
./dolphindb -home /path/to/dolphindb

# 3. 启动 RabbitMQ
rabbitmq-server
```

8.2 初始化数据库

```
// 在 DolphinDB GUI 中执行
run "</path>/imba-vnpy/script/dos/init.dos"
```

8.3 启动系统

终端1：启动 **vn.py** 主程序

```
python run.py
```

终端2：启动因子订阅 + 信号生成

```
python script/python/get_factors.py
```

8.4 验证流程

1. 在 **vn.py** 中连接 CTP Gateway（本项目使用Simsnow 7*24 模拟账号）
2. 订阅合约行情（本项目使用 IM2603）
3. 观察 DolphinDB 流表数据写入
4. 观察因子计算结果输出
5. 当因子值 > 0.7 或 < 0.3 时，观察信号生成和订单发送

九、技术难点与解决方案

9.1 流表列数不匹配问题

问题： tick_stream 有 23 列，但因子引擎只需要 4-6 列

解决： 在 handler 中使用 select 筛选列

```
def imbalanceHandler(mutable msg) {  
    selectedData = select symbol, datetime, bid_volume_1, ask_volume_1 from msg  
    engine.append!(selectedData)  
}
```

9.2 Python 订阅 batchSize 问题

问题： msgAsTable=True 时必须指定 batchSize

解决： 显式设置 batchSize=1, throttle=0.01

9.3 无价格信号的下单问题

问题： 信号不包含价格，但限价单需要价格

解决： 实现对手价获取逻辑

```
def _get_price(self, signal, use_ask):  
    tick = self.main_engine.get_tick(vt_symbol)  
    return tick.ask_price_1 if use_ask else tick.bid_price_1
```

9.4 MQ 队列不存在问题

问题： 消费者先于生产者启动时，队列不存在

解决： 消费者启动时主动声明队列

```
def _setup_queue(self):  
    self.channel.exchange_declare(exchange='trading_signals', exchange_type='direct', durable=True)  
    self.channel.queue_declare(queue='trading_signals', durable=True)  
    self.channel.queue_bind(queue='trading_signals', exchange='trading_signals', routing_key='s:
```

十、总结与展望

10.1 项目成果

通过本项目的开发，我完成了一个**端到端的高频量化交易系统**原型，涵盖了：

- 1. **数据层**：实时采集 + 流式存储 + 历史持久化
- 2. **计算层**：多因子实时计算 + 组合策略
- 3. **信号层**：阈值触发 + 可靠分发
- 4. **执行层**：对手价下单 + 异步执行

10.2 技术收获

领域	学习内容	掌握程度
DolphinDB	流表、分区表、响应式引擎、订阅机制	熟练
vn.py	事件驱动、Engine 扩展、Gateway 机制	熟练
RabbitMQ	生产者消费者模式、队列持久化	掌握
因子开发	有状态/无状态因子、因子组合	掌握

10.3 后续优化方向

- 1. **风控模块**：增加持仓限制、撤单逻辑
- 2. **因子回测**：利用历史数据验证因子有效性
- 3. **策略可视化**：Web 界面展示实时因子和信号
- 4. **多策略并行**：支持多个策略同时运行
- 5. **弹性扩展**：针对行情获取，数据库读写，消息队列，下单等考虑容灾/弹性扩展等高可用设计
- 6. **工程化拆解重构**：对项目进行工程化规范，对DDB的脚本文件进行模块化分解/调用等

许可证

本项目仅用于学习和研究目的。